

# Curso Libre de Configuración 2: Análisis de Datos

## Tarea 2 - Investigación 1

Daniela Natareno – `dnatareno@unis.edu.gt`

**Fecha de entrega:** 20 de septiembre del 2026

---

### 1. Generadores de Congruencia Lineal (LCG)

Es uno de los algoritmos más antiguos y conocidos, el cual produce una secuencia de números pseudoaleatorios calculados por medio de una ecuación lineal por partes discontinua. Su implementación es sencilla y rápida, sobre todo en hardware informático donde las operaciones modulares se resuelven directamente por las propias limitaciones físicas del hardware. [1]

Es Lineal debido a que utiliza como base la ecuación de primer grado  $y = mx + b$  y Congruencia ya que utiliza la operación modular, la cual puede tener el mismo residuo al dividir por otro número específico. [2]

#### Fundamento matemático

$$X_{n+1} = (a \times X_n + c) \bmod m$$

Donde:

- $X_n$  es el número aleatorio actual,
- $X_{n+1}$  es el nuevo número que se generará,
- $m$  es el módulo, y  $m > 0$ ,
- $a$  es el multiplicador, y  $0 < a < m$ ,
- $c$  es el incremento, y  $0 \leq c < m$ ,
- $X_0$  es el valor semilla, y  $0 \leq X_0 < m$ .

Esta es una ecuación recursiva donde se siguen los siguientes pasos:

1. Se inicia con un valor inicial obligatorio que es la semilla  $X_0$ .
2. Se toma  $X_0$ , luego se multiplica por  $a$ , se le suma  $c$  y se calcula el residuo al dividirlo por  $m$ . El resultado es  $X_1$ .
3. Para calcular el siguiente número,  $X_1$  es la nueva entrada de la fórmula para producir  $X_2$ . Y así sucesivamente para cada  $X$ .

## Normalización

Para mapear los valores enteros generados al intervalo  $[0, 1)$ , donde cada valor generado se divide entre el módulo  $m$ , obteniendo así la fórmula de normalización:

$$u_n = \frac{X_n}{m}$$

Y los valores normalizados cumplen con la condición  $0 \leq u_n < 1$  ya que  $0 \leq X_n \leq m - 1$ .

## Período

Los LCG tienen período, el cual es la cantidad de números únicos que puede generar el algoritmo antes de comenzar a repetir la misma secuencia. El período ( $p$ ) máximo es igual al módulo  $m$ , únicamente se puede alcanzar si cumple las condiciones del teorema de Hull-Dobell. [3][2]

## Teorema de Hull-Dobell

Para que un generador de congruencia lineal alcance un período completo igual al módulo  $m$ , deben cumplirse las siguientes condiciones:

- $c$  y  $m$  son primos relativos, es decir, su máximo común divisor es 1:

$$\gcd(c, m) = 1$$

- $a - 1$  es divisible por todos los factores primos del módulo  $m$ .
- Si  $m$  es divisible por 4, entonces  $a - 1$  también debe ser divisible por 4.

## Pseudocódigo

```
1 Inicio Algoritmo_LCG
2     Definir n, m, a, c, semilla, i, X como entero
3     Definir u como real
4     Ingresar m
5     Ingresar a
6     Ingresar c
7     Ingresar n
8     Ingresar semilla
9     X = semilla
10
11     Para i = 1 Hasta n Hacer
12         X = (a * X + c) Mod m
13         u = X/m
14         Escribir X y u
15     Fin Para
16 Fin Algoritmo_LCG
```

## Implementación

A partir del pseudocódigo se realizó la implementación del LCG y su normalización en Python.

Se utilizaron los siguientes parámetros:

| Parámetro             | Valor      |
|-----------------------|------------|
| Módulo m              | $2^{32}$   |
| Multiplicador a       | 1664525    |
| Incremento c          | 1013904223 |
| Semilla $X_0$         | 8102003    |
| Cantidad de valores n | 10,000     |

Los parámetros utilizados (menos semilla) fueron utilizados los de Numerical Recipes ([https://en.wikipedia.org/wiki/Linear\\_congruential\\_generator](https://en.wikipedia.org/wiki/Linear_congruential_generator))

La implementación almacena tanto los valores entero  $X_n$  como sus correspondientes valores normalizados  $u_n$ , los normalizados serán utilizados para realizar el análisis estadístico de la secuencia.

### a. Histograma de los valores generados

Se construyó un histograma utilizando los 10,000 valores normalizados generados por el LCG. El intervalo  $[0,1)$  fue dividido en 20 intervalos, por lo tanto, se espera una frecuencia cercana a 500 valores por intervalo.

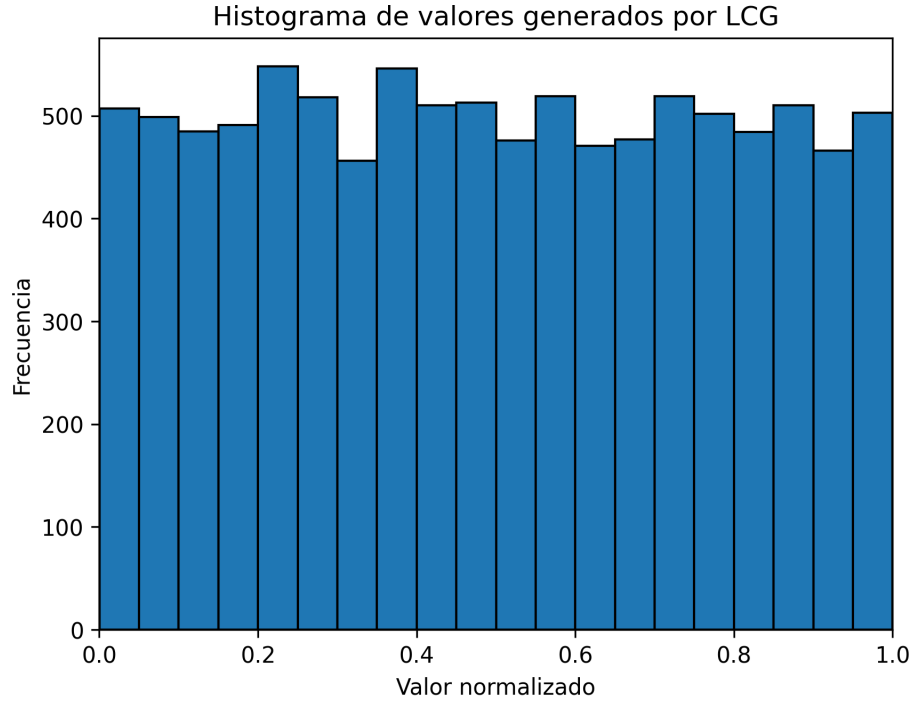


Figura 1: Histograma de valores normalizados generados por LCG.

El histograma muestra que las frecuencias se encuentran distribuidas aproximadamente alrededor de los 500 valores por intervalo, presentando variaciones entre las distintas barras.

Visualmente este comportamiento es compatible con una distribución aproximadamente uniforme.

## b. Pruebas de hipótesis

### b.1. Prueba de uniformidad

Para evaluar si los valores normalizados generados por el LCG presentan una distribución uniforme en el intervalo  $[0,1)$ , se aplicó la prueba de bondad de Kolmogorov-Smirnov. Para realizar la prueba se establecieron las siguientes hipótesis:

$H_0$ : Los valores generados siguen una distribución uniforme  $U(0,1)$

$H_1$ : Los valores generados no siguen una distribución uniforme  $U(0,1)$

Se utilizó un nivel de significancia de  $\alpha = 0,05$ .

La prueba de Kolmogorov-Smirnov compara la distribución acumulada de los valores generados con la distribución acumulada teórica esperada. El estadístico  $D$  representa la máxima diferencia observada entre ambas distribuciones.

La prueba de Kolmogorov-Smirnov obtuvo un estadístico  $D = 0,008304$  y un p-valor de 0,492974. Al utilizar un nivel de significancia de  $\alpha = 0,05$ , se observa que el p-valor es mayor que el nivel de significancia ( $0,492974 > 0,05$ ); por lo tanto, no se rechaza la hipótesis nula.

No existe evidencia estadística suficiente para afirmar que los valores generados por el LCG no siguen una distribución uniforme  $U(0,1)$ . Este resultado es consistente con el comportamiento observado previamente en el histograma.

## **b.2. Prueba de independencia serial**

Para evaluar la independencia serial de la secuencia generada por el LCG, se utilizó una prueba de autocorrelación de retardo 1, con el propósito de determinar si existe una relación lineal significativa entre cada valor  $u_i$  y el valor generado inmediatamente después  $u_{i+1}$ .

Se plantearon las siguientes hipótesis:

$$H_0 : \rho_1 = 0$$

$$H_1 : \rho_1 \neq 0$$

Donde  $\rho_1$  representa la correlación entre valores consecutivos de la secuencia. La hipótesis nula establece que no existe correlación lineal significativa de retardo 1, mientras que la hipótesis alternativa establece que sí existe.

Se utilizó nuevamente un nivel de significancia de  $\alpha = 0,05$ .

La prueba de autocorrelación de retardo 1 obtuvo un coeficiente de correlación de  $r = -0,001564$  y un p-valor de  $0,87575$ . El coeficiente obtenido se encuentra cercano a cero, lo que indica una correlación lineal muy baja entre valores consecutivos de la secuencia.

Debido a que el p-valor obtenido es mayor que el nivel de significancia establecido ( $0,875754 > 0,05$ ), no se rechaza la hipótesis nula. Por lo tanto, no existe evidencia estadística suficiente de una correlación lineal significativa entre valores consecutivos. Este resultado es compatible con independencia serial de retardo 1, aunque no demuestra independencia en todas sus posibles formas.

## **c. Prueba espectral visual**

Se construyeron ternas de la forma  $(u_i, u_{i+1}, u_{i+2})$ , para poder visualizar la existencia de posibles patrones entre valores generados por el generador LCG.

Cada terna fue representada como un punto dentro del espacio tridimensional  $[0, 1]^3$ . Esta representación permite observar posibles estructuras, agrupamientos o patrones entre valores consecutivos que podrían no ser evidentes mediante el histograma o las pruebas estadísticas realizadas anteriormente.

En la representación tridimensional obtenida, los puntos correspondientes a las ternas consecutivas  $(u_i, u_{i+1}, u_{i+2})$  parecen distribuirse a lo largo del cubo  $[0, 1]^3$ , sin observarse visualmente planos, bandas o regiones vacías claramente definidas.

Este comportamiento sugiere que para los parámetros y la cantidad de valores utilizados, no se evidencia mediante esta representación una estructura tridimensional marcada entre valores consecutivos. Sin embargo, al tratarse de un LCG, la secuencia es determinista y

## Prueba espectral visual del LCG

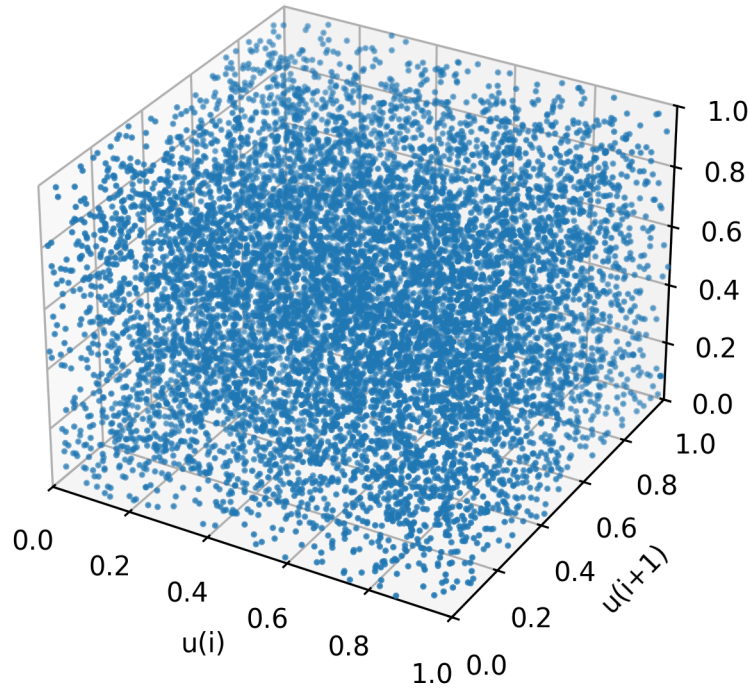


Figura 2: Prueba espectral visual del Generador de Congruencia Lineal.

posee una estructura matemática finita, por lo que la ausencia de patrones evidentes en esta gráfica no implica independencia completa ni aleatoriedad verdadera.

El módulo utilizado es  $m = 2^{32}$  y bajo las condiciones de período completo del LCG empleado, la secuencia puede alcanzar un período de  $2^{32}$  valores. Debido a que en este experimento únicamente se generaron 10,000 valores, se está analizando una fracción muy pequeña del período total. Los resultados obtenidos muestran un comportamiento favorable en las pruebas realizadas; pero, un período largo y una buena apariencia visual no garantizan por sí solos una alta calidad estadística en todas las dimensiones.

### Verificación del período

Para comprobar que los parámetros seleccionados permiten alcanzar el período completo del generador, se verificaron las condiciones establecidas por el teorema de Hull-Dobell. Para  $m = 2^{32}$ ,  $a = 1664525$  y  $c = 1013904223$ , se obtuvo:

$$\gcd(c, m) = 1$$

$$(a - 1) \bmod 2 = 0$$

$$(a - 1) \bmod 4 = 0$$

Debido a que  $m = 2^{32}$  tiene únicamente a 2 como factor primo y las tres condiciones se cumplen, el generador puede alcanzar un período completo igual al módulo:

$$T = 2^{32} = 4,294,967,296$$

Por lo tanto, los 10,000 valores utilizados en las pruebas representan solamente una fracción del período completo del generador.

La ausencia de patrones claramente visibles en la gráfica es consistente con el comportamiento observado en las pruebas anteriores. Además, el generador posee un período completo de  $2^{32}$ , mientras que el análisis utiliza únicamente 10,000 valores. Sin embargo, un período largo y resultados favorables en estas pruebas no garantizan por sí solos la ausencia de estructura o dependencias en dimensiones superiores.

## 2. Método de cuadrados medio (Middle-square method)

Método inventado por John von Neumann, descrito en 1949. Este algoritmo es para generar números pseudoaleatorios, dependiendo de su aplicación es la definición que se le da. Su período suele ser muy corto siendo una debilidad ya que se repite indefinidamente los mismos números, entrando en un bucle.<sup>6</sup>

### Fundamento matemático

$$X_{n+1} = \left\lfloor \frac{X_n^2}{10^{\frac{D}{2}}} \right\rfloor \bmod 10^D$$

Donde:

- $X_n$  es el valor actual de la secuencia,
- $X_n^2$  es el valor actual al cuadrado,
- $X_{n+1}$  siguiente valor pseudoaleatorio de la secuencia,
- $X_0$  es el valor semilla,
- $D$  número de dígitos a extraer de cada valor,
- $10^{\frac{D}{2}}$  permite eliminar la mitad de los dígitos  $D$  de la derecha de  $X_n^2$  para aislar los dígitos centrales,

- $\lfloor \rfloor$  función floor (piso), parte entera de un resultado (elimina parte decimal),
- $\text{mod } 10^D$  conserva los últimos  $D$  dígitos del resultado anterior.

Para generar la secuencia se deben seguir los siguientes pasos<sup>7</sup>:

1. Elegir  $X_0$  de  $D$  dígitos.
2. Elevar el  $X_0$  para obtener un número con  $2D$  dígitos, si el resultado es menos de  $2D$  es necesario que se rellene con ceros a la izquierda.
3. Extraer los  $D$  dígitos centrales para formar el siguiente número de la secuencia.
4. Repetir el proceso la cantidad de veces sea necesario, pero, en las siguientes rondas ya no se utiliza  $X_0$  sino  $X_{n+1}$  generado en la ronda anterior siendo  $X_n$ .

## Normalización

Al obtener el valor pseudoaleatorio  $X_{n+1}$ , se normaliza para obtener un valor dentro del intervalo  $[0, 1)$ . Para ello se utiliza:

$$r_n = \frac{X_{n+1}}{10^D}$$

- $X_{n+1}$  valor obtenido por los dígitos centrales de  $X_n^2$ ,
- $r_n$  es el valor normalizado correspondiente al número generado,
- $10^D$  convierte el número  $X_{n+1}$  en un valor entre 0 y 1, siendo  $0 \leq r_n < 1$ .

## Pseudocódigo

```

1  Inicio Algoritmo_MSM
2      Definir semilla, digitos, generado, floor, X, i como entero
3      Definir r como real
4      Ingresar semilla
5      Ingresar digitos
6      Ingresar n
7      X = semilla
8      Para i = 1 Hasta i = n Hacer
9          Floor = truncar ((X^2)/(10^(digitos/2)))
10         X = floor mod 10^digitos
11         r = X/10^digitos
12         Escribir X y r
13     Fin Para
14 Fin Algoritmo_MSM

```

## Implementación

Se implementó el pseudocódigo realizado en Python para generar la secuencia de MSM, se observó que para la semilla inicial  $X_0 = 8103$ , los valores disminuyeron progresivamente hasta alcanzar  $X = 0000$  en la iteración 11. A partir de este estado, el generador permanece en cero, debido a que elevar cero al cuadrado y extraer sus dígitos centrales produce nuevamente cero.

Este comportamiento evidencia una de las limitaciones del método de cuadrados medios, dependiendo de la semilla utilizada, la secuencia puede degenerar rápidamente hacia un estado absorbente o presentar ciclos cortos.

### a. Histograma de los valores generados

Para observar la distribución de los valores generados mediante el método de cuadrados medios, se construyó un histograma a partir de los 10,000 valores normalizados. Al igual que en el análisis anterior, los valores fueron agrupados en 20 intervalos dentro de  $[0, 1)$ .

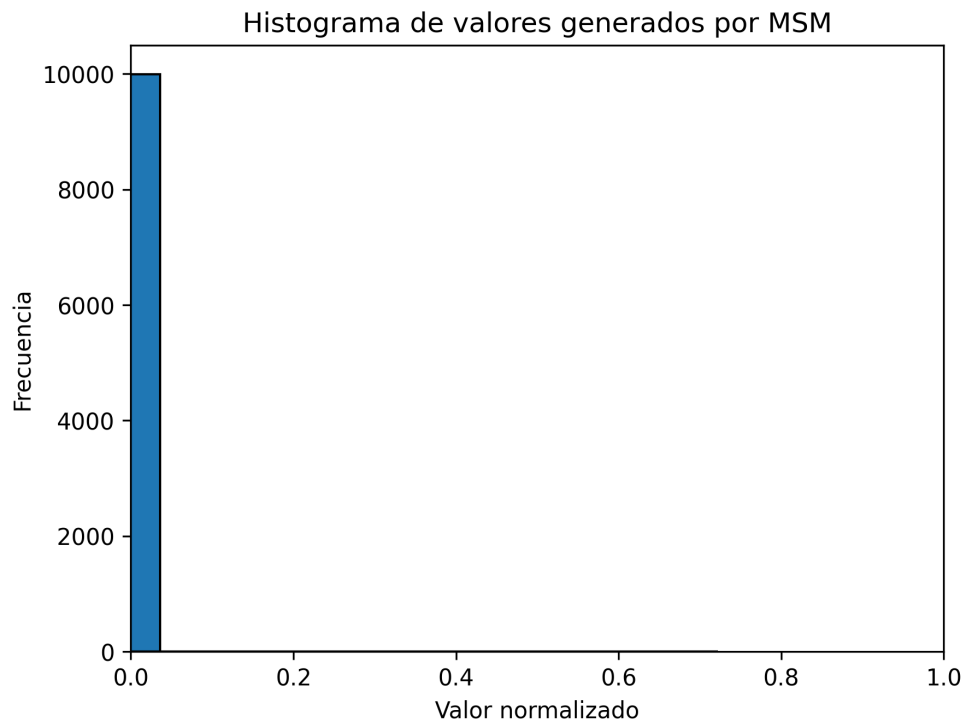


Figura 3: Histograma de valores normalizados generados mediante el método de cuadrados medios.

El histograma presenta una concentración prácticamente total de los valores en el primer intervalo cercano a cero, mientras que el resto del intervalo  $[0,1)$  presenta una frecuencia mínima o nula. Este comportamiento se debe a que para la semilla  $X_0 = 8103$ , la secuencia alcanza el valor 0000 en la iteración 11 y a partir de ese punto, permanece en dicho estado.

Por lo tanto, los valores generados no presentan visualmente una distribución uniforme. El resultado evidencia una de las principales debilidades del método de cuadrados medios, su comportamiento depende fuertemente de la semilla y puede degenerar rápidamente en ciclos cortos o estados absorbentes.

## **b. Pruebas de hipótesis**

### **b.1. Prueba de uniformidad**

Para evaluar si los valores normalizados generados mediante el método de cuadrados medios presentan una distribución uniforme en el intervalo  $[0,1)$ , se aplicó la prueba de bondad de ajuste de Kolmogorov-Smirnov (K-S).

Se establecieron las siguientes hipótesis:

$H_0$ : Los valores generados siguen una distribución uniforme  $U(0,1)$

$H_1$ : Los valores generados no siguen una distribución uniforme  $U(0,1)$

Utilizando un nivel de significancia de  $\alpha = 0,05$ . El estadístico  $D$  representa la máxima diferencia entre la distribución acumulada de los valores generados y la distribución acumulada teórica de una distribución uniforme.

La prueba de K-S obtuvo un estadístico  $D = 0,999000$  y un p-valor reportado de 0,000000. Debido a que el p-valor es menor que el nivel de significancia establecido, se rechaza la hipótesis nula.

Por lo tanto, existe evidencia estadística suficiente para concluir que la secuencia generada no sigue una distribución uniforme  $U(0,1)$ . El elevado valor del estadístico  $D$ , cercano a 1, evidencia una gran discrepancia entre la distribución observada y la distribución uniforme teórica. Este resultado es consistente con el histograma y con el hecho de que la secuencia alcanza el estado 0000 en la iteración 11 y permanece en él durante las iteraciones posteriores.

### **b.2. Prueba de independencia serial**

Para evaluar la independencia serial de la secuencia generada mediante el método de cuadrados medios, se utilizó una prueba de autocorrelación de retardo 1, comparando cada valor  $u_i$  con el valor generado inmediatamente después  $u_{i+1}$ .

Se establecieron las siguientes hipótesis:

$$H_0 : \rho_1 = 0$$

$$H_1 : \rho_1 \neq 0$$

Donde  $\rho_1$  representa la correlación entre valores consecutivos. La hipótesis nula establece que no existe una correlación lineal significativa de retardo 1, mientras que la hipótesis alternativa establece que sí existe.

Se utilizó un nivel de significancia de  $\alpha = 0,05$ .

La prueba de autocorrelación de retardo 1 obtuvo un coeficiente de correlación de  $r = 0,502119$  y un p-valor reportado de 0,000000. El coeficiente obtenido indica una correlación lineal positiva moderada entre valores consecutivos de la secuencia.

Debido a que el p-valor es menor que el nivel de significancia establecido, se rechaza la hipótesis nula. Por lo tanto, existe evidencia estadística suficiente de una correlación lineal significativa entre valores consecutivos.

Este resultado es consistente con la degeneración observada en la secuencia, la cual alcanza el valor 0000 en la iteración 11 y posteriormente permanece en dicho estado. Por lo tanto, para la semilla utilizada, el método de cuadrados medios presenta una baja calidad como generador pseudoaleatorio.

### c. Prueba espectral visual

De manera complementaria, se realizó una representación tridimensional de las ternas consecutivas  $(u_i, u_{i+1}, u_{i+2})$  generadas mediante el método de MSM.

Prueba espectral visual del MSM

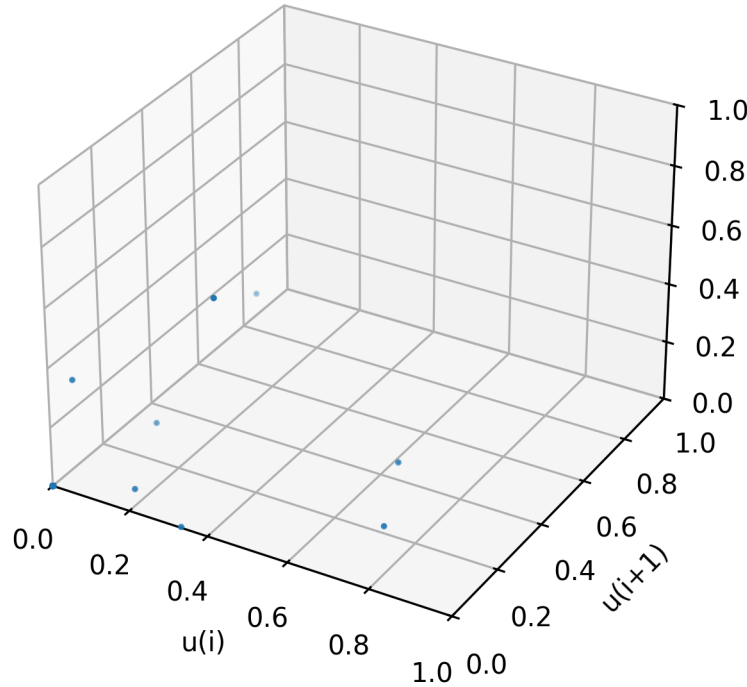


Figura 4: Prueba espectral visual del método de cuadrados medios.

En la representación tridimensional se observa una cantidad muy reducida de puntos distintos en comparación con el número total de valores generados. Esto se debe a que la secuencia producida con la semilla  $X_0 = 8103$  alcanza el valor cero en la iteración 11.

A partir de este punto, los valores posteriores permanecen en cero, provocando que la mayoría de las ternas consecutivas correspondan al mismo punto  $(0, 0, 0)$  y se superpongan en la representación gráfica. Por esta razón, los puntos no ocupan uniformemente el cubo  $[0, 1]^3$ .

Este comportamiento evidencia el corto período obtenido por la semilla utilizada y la baja calidad de la secuencia generada. El método de cuadrados medios puede caer en estados absorbentes o ciclos cortos dependiendo de la semilla, reduciendo considerablemente la cantidad de valores distintos generados.

### 3. Mersenne Twister

El generador Mersenne Twister fue desarrollado en 1997 por Makoto Matsumoto y Takuji Nishimura. Es un generador de números pseudoaleatorios siendo uno de los más utilizados en simulaciones computacionales.<sup>11</sup>

Su nombre proviene de la longitud de su período, teniendo relación con su número primo de Mersenne (número primo que tiene la forma  $M_p = 2^p - 1$ , donde  $p$  es un número primo<sup>12</sup>).

Entre las variantes del algoritmo se encuentran MT19937, diseñada para trabajar con palabras de 32 bits, y MT19937-64, adaptada para palabras de 64 bits. Ambos poseen un periodo de  $2^{19937} - 1$

### Fundamento matemático

$$x_{k+n} = x_{k+m} \oplus (x_k^u | x_{k+1}^l)A$$

Donde:

- $x_k$  palabra del estado interno del generador,
- $k$  posición del valor que se utiliza dentro de la secuencia o estado,
- $n$  tamaño del estado utilizado en la recurrencia, para MT19937 mantiene 624 palabras de 32 bits como estado interno,
- $m$  desplazamiento utilizado para seleccionar otra palabra del estado,
- $x_{k+m}$  es el valor del estado interno ubicado  $m$  posiciones delante de  $x_k$ ,
- $\oplus$  representa la operación XOR a nivel de bits, utilizado para combinar  $x_{k+m}$  con  $A$ ,
- $x_k^u$  representa la parte superior de  $x_k$ , conserva su bit más significativo,
- $x_{k+1}^l$  representa la parte inferior de  $x_{k+1}$ , formado por sus 31 bits menos significativos.
- $|$  operación OR a nivel de bits, por lo tanto, combina la parte superior de  $x_k$  con la inferior de  $x_{k+1}$ , formando un nuevo valor de 32 bits.

- $A$  representa la transformación de torsión (twist) aplicada al valor formado por los bits de la parte superior  $x_k$  y la inferior  $x_{k+1}$ .

Transformación de torsión (twist): para realizar esta transformación, se define  $y = x_k^u | x_{k+1}^1$ . Sobre  $y$  se aplica la transformación  $A$ .

$$yA = \begin{cases} y \gg 1, & \text{si el bit menos significativo de } y = 0, \\ (y \gg 1) \oplus a, & \text{si el bit menos significativo de } y = 1. \end{cases}$$

Donde:

- $y$  es el valor de 32 bits, obtenido al combinar  $x_k$  y  $x_{k+1}$ ,
- $\gg 1$  desplaza los bits de  $y$  una posición a la derecha,
- $\oplus$  operación XOR a nivel de bits,
- $a$  es una constante utilizada en la transformación.

Tempering: al actualizar el estado interno por medio de la transformación twist, el valor pasa por una transformación final denominada tempering. Se realiza un desplazamiento y opera con AND y XOR a nivel de bits para mejorar las propiedades estadísticas del valor que entregará como salida. Las operaciones que utiliza son:

$$y = x$$

$$y = y \oplus ((y \gg u) \wedge d)$$

$$y = y \oplus ((y \ll s) \wedge b)$$

$$y = y \oplus ((y \ll t) \wedge c)$$

$$y = y \oplus (y \gg l)$$

Donde:

- $x$  es la palabra obtenida del estado interno,
- $y$  es el valor que se transforma hasta obtener la salida pseudoaleatoria,
- $\oplus$  representa la operación XOR a nivel de bits,
- $\wedge$  representa la operación AND a nivel de bits,
- $\gg$  desplazamiento de bits hacia la derecha,

- $\ll$  desplazamiento de bits hacia la izquierda,
- $u, s, l, t$  indican la cantidad de posiciones que se desplazan los bits,
- $d, b, c$  máscaras de bits utilizadas durante las transformaciones.

## Normalización

El MT19937 genera valores enteros de 32 bits, estos se encuentran en el intervalo  $0 \leq y \leq 2^{32} - 1$ . Para normalizarlo dentro del intervalo  $[0, 1)$ , se debe dividir entre  $2^{32}$ .

$$r = \frac{y}{2^{32}}$$

Donde:

- $y$  es el valor entero de 32 bits obtenido después de tempering,
- $2^{32}$  cantidad de valores diferentes que puede representarse con 32 bits,
- $r$  valor normalizado que cumple  $0 \leq r < 1$ .

## Pseudocódigo

```

1  Inicio Algoritmo_MT19937
2      Definir semilla, n, i, k como entero
3      Definir estado[624] como entero
4      Definir y, x, r como real
5      Ingresar semilla
6      Ingresar n
7      estado[0] = semilla
8
9      Para i = 1 Hasta 623 Hacer
10         estado[i] = (1812433253 * estado[i-1] XOR (estado[i-1] >>
11             30)) + i) Mod 2^32
12     Fin Para
13
14     Para k = 0 Hasta n-1 Hacer
15         y = (estado[k Mod 624] AND 0x80000000) OR (estado[(k+1)
16             Mod 624] AND 0x7FFFFFFF)
17
18         Si el bit menos significativo de y = 0 Entonces
19             x = y >> 1
20         Sino
21             x = (y >> 1) XOR 0x9908B0DF
22         Fin Si

```

```

22         estado[k Mod 624] = estado[(k+397) Mod 624] XOR x
23         y = estado[k Mod 624]
24         y = y XOR ((y >> 11) AND 0xFFFFFFFF)
25         y = y XOR ((y << 7) AND 0x9D2C5680)
26         y = y XOR ((y << 15) AND 0xEFC60000)
27         y = y XOR (y >> 18)
28         r = y / 2^32
29         Escribir y y r
30     Fin Para
31 Fin Algoritmo_MT19937

```

## Implementación

Para la implementación del generador Mersenne Twister se utilizó la variante MT19937 de 32 bits, desarrollada a partir del pseudocódigo definido anteriormente.

El algoritmo utiliza un vector de estado interno compuesto por 624 palabras de 32 bits. Debido a que MT 19937 genera valores enteros de 32 bits, cada valor de salida  $y_i$  fue normalizado mediante  $r_i = \frac{y_i}{2^{32}}$  obteniendo así valores en el intervalo [0,1).

Parámetros utilizados:

| Parámetro                 | Valor      |
|---------------------------|------------|
| $w$                       | 32         |
| $n$ del estado            | 624        |
| $m$                       | 397        |
| $r$                       | 31         |
| $a$                       | 0x9908B0DF |
| Semilla                   | 8103       |
| Valores generados ( $n$ ) | 10,000     |

### a. Histograma de los valores generados

Se construyó un histograma utilizando los 10,000 valores normalizados en el intervalo [0,1). Los valores fueron agrupados en 20 intervalos para observar la distribución de sus frecuencias.

El histograma muestra que los valores generados se distribuyen a lo largo de todo el intervalo. Al utilizar 20 intervalos para 10,000 observaciones, se esperaría aproximadamente una frecuencia de 500 valores por intervalo bajo una distribución uniforme.

Las frecuencias observadas presentan pequeñas variaciones alrededor de este valor esperado. Visualmente, la secuencia presenta un comportamiento compatible con una distribución uniforme.

Sin embargo, el histograma constituye únicamente una evaluación visual, por lo que es necesario aplicar una prueba estadística de bondad de ajuste para determinar si existe

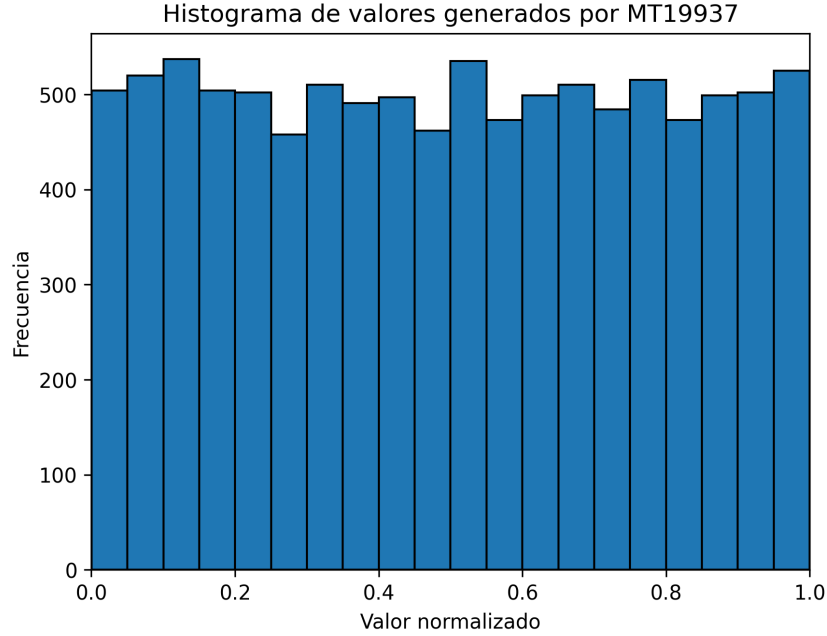


Figura 5: Histograma de valores normalizados generados mediante MT19937.

evidencia significativa en contra de la uniformidad.

## b. Pruebas de hipótesis

### b.1. Prueba de uniformidad

Para evaluar formalmente la uniformidad de los valores normalizados generados mediante MT19937, se aplicó la prueba de bondad de ajuste de Kolmogorov-Smirnov, comparando la distribución empírica de los valores generados con una distribución  $U(0,1)$ .

Se establecieron las siguientes hipótesis:

$H_0$ : Los valores generados siguen una distribución uniforme  $U(0,1)$

$H_1$ : Los valores generados no siguen una distribución uniforme  $U(0,1)$

Se utilizó un nivel de significación de  $\alpha = 0,05$ . El estadístico  $D$  representa la máxima diferencia entre la función de distribución acumulada de los valores generados y la función de distribución acumulada de la distribución uniforme teórica.

La prueba de K-S obtuvo un estadístico  $D = 0,007441$  y un p-valor de 0,634127. Debido a que el p-valor es mayor que el nivel de significancia establecido, no se rechaza la hipótesis nula.

Por lo tanto, no existe evidencia estadística suficiente para afirmar que los valores generados por MT19937 difieren de una distribución uniforme  $U(0,1)$ . Este resultado es consistente con el histograma obtenido anteriormente, en el cual las frecuencias se distribuyeron aproximadamente de manera uniforme a lo largo del intervalo  $[0,1)$ .

## b.2. Prueba de independencia serial

Para evaluar la independencia serial de la secuencia generada mediante MT19937, se aplicó una prueba de autocorrelación de retardo 1, comparando cada valor  $u_i$  con el valor generado inmediatamente después  $u_{i+1}$ .

Se establecieron las siguientes hipótesis:

$$H_0 : \rho_1 = 0$$

$$H_1 : \rho_1 \neq 0$$

Donde  $\rho_1$  representa la correlación entre valores consecutivos. La hipótesis nula establece que no existe una correlación lineal significativa de retardo 1, mientras que la hipótesis alternativa establece que sí existe.

Se utilizó un nivel de significancia de  $\alpha = 0,05$ .

La prueba de autocorrelación de retardo 1 obtuvo un coeficiente de correlación de  $r = 0,005389$  y un p-valor de 0,590031. Debido a que el p-valor es mayor que el nivel de significancia establecido, no se rechaza la hipótesis nula.

Por lo tanto, no existe evidencia estadística suficiente para afirmar que existe una correlación lineal significativa entre valores consecutivos de la secuencia generada. Además, el coeficiente de correlación obtenido es cercano a cero, lo cual indica una asociación lineal prácticamente nula para la muestra analizada.

Este resultado, junto con la prueba de uniformidad y el histograma, muestra un comportamiento estadístico favorable de MT19937 para los 10,000 valores analizados.

## c. Prueba espectral visual

De manera complementaria, se realizó una representación tridimensional de las ternas consecutivas  $(u_i, u_{i+1}, u_{i+2})$  generadas por MT19937. Cada terna fue representada como un punto dentro del cubo  $[0,1]^3$ , con el propósito de identificar visualmente posibles estructuras, agrupamientos, regiones vacías o patrones entre valores consecutivos.

En la representación, los puntos se distribuyen a lo largo del cubo. No se observan concentraciones, planos, bandas o regiones vacías claramente definidas, por lo tanto no se evidencia visualmente una estructura marcada entre tres valores consecutivos para la muestra analizada.

Este resultado es consistente con las pruebas realizadas anteriormente, en las cuales no se encontró evidencia estadística en contra de la uniformidad ni de correlación lineal significativa de retardo 1.

MT19937 posee un período teórico de  $T = 2^{19937} - 1$ , el cual es considerablemente mayor que los 10,000 valores utilizados en este experimento. Por lo tanto, la muestra analizada representa una fracción extremadamente pequeña de su período y no se espera observar repetición de la secuencia durante el experimento.

### Prueba espectral visual del MT19937

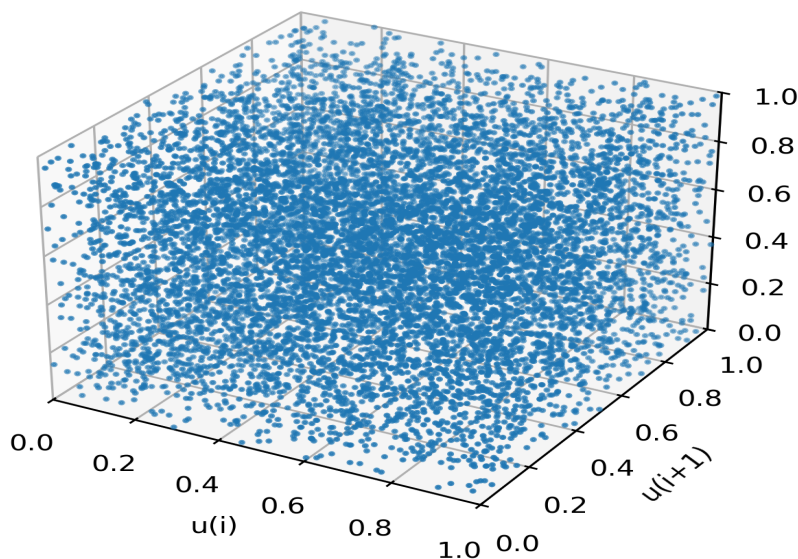


Figura 6: Prueba espectral visual de MT19937.

En conjunto, el largo período y los resultados favorables obtenidos en el histograma, la prueba de uniformidad, la prueba de independencia serial y la representación tridimensional indican un buen comportamiento estadístico de MT19937 para la muestra analizada. Sin embargo, un período extenso y resultados favorables en estas pruebas no constituyen por sí solos una demostración de aleatoriedad ni garantizan el comportamiento del generador frente a todas las pruebas estadísticas posibles.

## 4. Blum Blum Shub

Generador de números pseudoaleatorios propuesto por Lenore Blum, Manuel Blum y Michael Shub en 1986, por ello su nombre Blum Blum Shub (BBS).

Es un generador pseudoaleatorio criptográficamente seguro, su seguridad se basa en la dificultad computacional de factorizar números enteros grandes.

### Fundamento matemático

#### 1. Selección de los números primos

Se debe seleccionar dos números primos distintos, siendo  $p$  y  $q$  primos de Blum, por lo que deben cumplir lo siguiente:

$$p \equiv 3 \pmod{4}$$

$$q \equiv 3 \pmod{4}$$

Quiere decir que, al dividir  $p$  y  $q$  dentro de 4, deben dar de residuo 3.

## 2. Cálculo del módulo

Al seleccionar  $p$  y  $q$  se calcula el módulo  $M = p \bullet q$ . Donde,

- $p$  es el primer número primo,
- $q$  es el segundo número primo, y
- $M$  es el módulo utilizado por el generador.

Para criptografía se utilizan primos considerablemente grandes.

## 3. Selección de la semilla

Se debe elegir una semilla  $s$  que debe ser coprima con  $M$ . Por lo tanto, su máximo común divisor de ambos debe ser 1,  $\gcd(s, M) = 1$ .

A partir de aquí, se obtiene el estado inicial por medio de:

$$x_0 = s^2 \bmod M$$

donde:

- $s$  es la semilla del generador,
- $x_0$  es el estado inicial, y
- $M$  es el módulo.

## 4. Generación de la secuencia

A partir del estado inicial, los siguientes estados se generan utilizando recurrencia, cada nuevo estado se utiliza para calcular el siguiente:

$$x_{n+1} = x_n^2 \bmod M$$

donde:

- $x_n$  es el estado actual,
- $x_{n+1}$  es el siguiente estado, y
- $M$  es el módulo  $p \bullet q$ .

## 5. Extracción de los bits

Originalmente el generador BBS es un generador de Bits, por lo que, después de obtener un nuevo estado, puede extraerse su bit menos significativo (LSB), utilizando:

$$b_n = x_{n+1} \bmod 2$$

Por lo tanto  $b_n \in \{0,1\}$

Si  $x_{n+1}$  es par, el bit obtenido es 0, sino es 1.

## 6. Construcción de números a partir de los bits

El bit producido por cada iteración se puede agrupar consecutivamente para formar un número entero  $R$ .

Se puede construir de la siguiente manera

$$R = \sum_{j=1}^k b_j 2^{k-j}$$

Donde:

- $R$  es el número entero resultante,
- $\sum_{j=1}^k$  sumar todos los valores calculados desde que  $j$  es 1 y llega al número total de bits  $k$ ,
- $b_j$  es el bit individual, su valor puede ser 0 o 1,
- $2^{k-j}$  es el peso posicional que se le asigna a  $b_j$  según el orden en que se generó
  - Para el primer bit generado se coloca a la izquierda, siendo el bit más significativo MSB.
  - Para el último bit generado el peso es  $2^0 = 1$  colocándolo a la derecha (bit menos significativo LSB).

## 7. Normalización

Al construir  $R$ , se puede normalizar al intervalo  $[0, 1)$  por medio de:

$$u = \frac{R}{2^k}$$

Ya que  $0 \leq R \leq 2^k - 1$  y se obtiene  $0 \leq u < 1$

## Pseudocódigo

```
1  Inicio Algoritmo_BBS
2      Definir p, q, M, semilla, bit, X, R, I, j, n, k como entero
3      Definir u como real
4
5      Repetir
6          Ingresar p
7          Si (p no es primo) O (p Mod 4 != 3) Entonces
8              Escribir "p debe ser primo y congruente con 3 modulo
9                  4"
10             Fin Si
11         Hasta que p sea primo Y (p Mod 4 = 3)
12
13     Repetir
14         Ingresar q
15         Si (q no es primo) O (q Mod 4 != 3) O (q = p) Entonces
16             Escribir "q debe ser primo, distinto de p y congruente
17                 con 3 modulo 4"
18         Fin Si
19     Hasta que q sea primo Y (p Mod 4 = 3) Y (q != p)
20
21     M = p * q
22
23     Repetir
24         Ingresar semilla
25         Si semilla = 0 O semilla = 1 Entonces
26             Escribir "La semilla no puede ser 0 ni 1"
27         Sino
28             Si MCD (semilla, M) != 1 Entonces
29                 Escribir "La semilla debe ser coprime con M"
30             Fin Si
31         Fin Si
32     Hasta Que (semilla != 0) Y (semilla != 1) Y (MCD(semilla, M) =
33         1)
34
35     Ingresar n
36     Ingresar k
37     X = (semilla^2) Mod M
38
39     Para i = 1 Hasta n Hacer
40         R = 0
41         Para j = 1 Hasta k Hacer
42             X = (X^2) Mod M
43             bit = X Mod 2
```

```

41         R = (R * 2) + bit
42     Fin Para
43     u = R / (2^k)
44     Escribir R y u
45 Fin Para
46 Fin Algoritmo_BBS

```

## Implementación

Se desarrolló el algoritmo a partir del pseudocódigo definido. El generador requiere dos números primos distintos  $p$  y  $q$ , ambos congruentes con  $3 \bmod 4$ . Por practicidad, se eligieron números pequeños para  $p$  y  $q$ .

### a. Histograma de los valores generados

Para analizar visualmente la distribución de los valores generados por Blum Blum Shub, se construyó un histograma utilizando los 10,000 valores normalizados en el intervalo  $[0,1)$ . Los valores fueron agrupados en 20 intervalos para observar la distribución de sus frecuencias. Bajo una distribución uniforme se esperaría aproximadamente una frecuencia de 500 observaciones por intervalo.

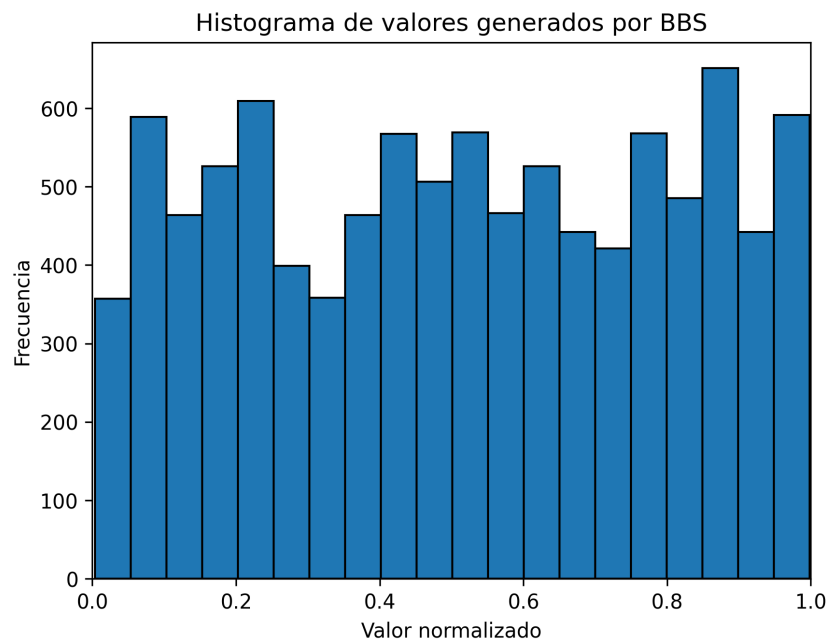


Figura 7: Histograma de valores normalizados generados mediante Blum Blum Shub.

En la gráfica se observan intervalos con frecuencias considerablemente inferiores o superiores a este valor esperado. Este generador presenta mayores irregularidades para la confi-

guración utilizada.

## b. Pruebas de hipótesis

### b.1. Prueba de uniformidad

Para evaluar formalmente la uniformidad de los valores generados mediante BBS, se aplicó la prueba de Kolmogorov-Smirnov, utilizando un nivel de significancia de  $\alpha = 0,05$ .

Se establecieron las siguientes hipótesis:

$H_0$ : Los valores generados siguen una distribución uniforme  $U(0,1)$

$H_1$ : Los valores generados no siguen una distribución uniforme  $U(0,1)$

La prueba de K-S obtuvo un estadístico  $D = 0,031727$  y un p-valor menor que  $3,522405 \times 10^9$ . Debido a que el p-valor es menor que el nivel de significancia establecido, se rechaza la hipótesis nula.

Por lo tanto, existe evidencia estadística de que los valores generados mediante BBS, con la configuración utilizada, no siguen una distribución uniforme  $U(0,1)$ . Este resultado es consistente con las irregularidades observadas previamente en el histograma.

El resultado corresponde específicamente a los parámetros utilizados en el experimento:

$$p = 383, \quad q = 503, \quad \text{semilla} = 8103 \text{ y } k = 16$$

### b.2. Prueba de independencia serial

Para evaluar la independencia serial de la secuencia generada mediante el método de BBS, se utilizó una prueba de autocorrelación de retardo 1, comparando cada valor  $u_i$  con el valor generado inmediatamente después  $u_{i+1}$ .

Se establecieron las siguientes hipótesis:

$$H_0 : \rho_1 = 0$$

$$H_1 : \rho_1 \neq 0$$

Donde  $\rho_1$  representa la correlación entre valores consecutivos. La hipótesis nula establece que no existe una correlación lineal significativa de retardo 1, mientras que la hipótesis alternativa establece que sí existe.

Se utilizó un nivel de significancia de  $\alpha = 0,05$ .

La prueba de autocorrelación de retardo 1 obtuvo un coeficiente de correlación de  $r = -0,068316$  y un p-valor de  $8,011011 \times 10^{12}$ . Debido a que el p-valor es menor que el nivel de significancia establecido, se rechaza la hipótesis nula.

Por lo tanto, existe evidencia estadística de correlación lineal entre valores consecutivos para la secuencia generada mediante BBS con la configuración utilizada. Sin embargo, el coeficiente de correlación obtenido es negativo y de pequeña magnitud, por lo que la asociación lineal detectada es débil.

Al igual que en la prueba de uniformidad, este resultado corresponde específicamente a los parámetros utilizados en el experimento y no debe interpretarse como una propiedad general del algoritmo BBS.

### c. Prueba espectral visual

De manera complementaria, se realizó una representación tridimensional de las ternas consecutivas  $(u_i, u_{i+1}, u_{i+2})$  generadas por BBS. Cada terna fue representada como un punto dentro del cubo  $[0,1]^3$ , con el propósito de identificar visualmente posibles estructuras, agrupamientos, regiones vacías o patrones entre valores consecutivos.

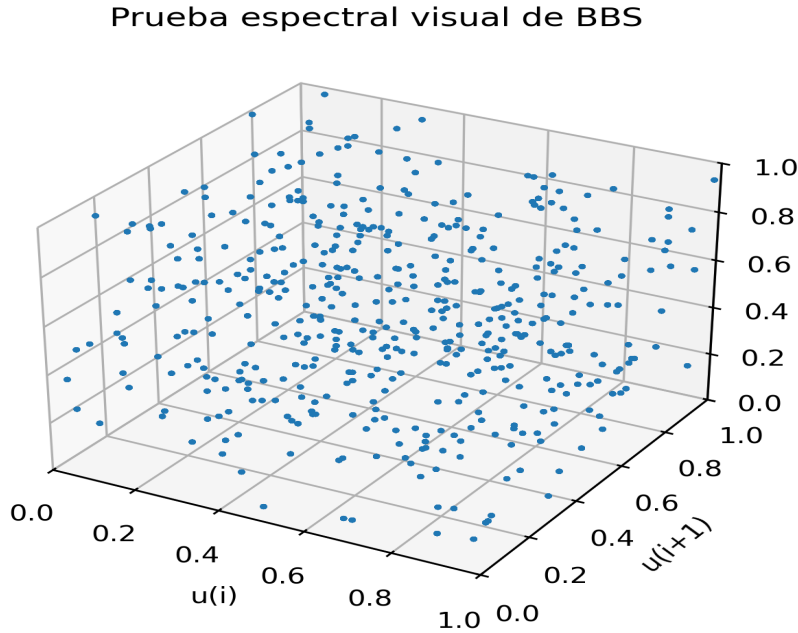


Figura 8: Prueba espectral visual de Blum Blum Shub.

En la representación tridimensional, los puntos aparecen distribuidos en distintas regiones del cubo  $[0,1]^3$ , sin observarse una concentración en un único punto ni una estructura de planos claramente definida. Sin embargo, se observa una cantidad reducida de posiciones distintas en comparación con el número total de ternas representadas.

Para comprobar este comportamiento, se contabilizaron las ternas generadas. De un total de 9,998 ternas, únicamente 475 fueron distintas, mientras que 9,523 correspondieron a repeticiones. Esto significa que aproximadamente el 95.25% de las ternas se repiten y se superponen en la representación tridimensional.

Este comportamiento evidencia una repetición considerable de la secuencia para la configuración utilizada y puede relacionarse con el tamaño reducido de los parámetros seleccionados. El módulo utilizado,  $M = 192649$ , proporciona un espacio de estados considerablemente menor al que se emplearía en una aplicación criptográfica real.

Este resultado complementa las pruebas estadísticas anteriores, en las cuales se encontró evidencia en contra de la uniformidad y una correlación lineal débil pero estadísticamente significativa entre valores consecutivos. Los resultados corresponden específicamente a la configuración experimental utilizada y no deben interpretarse como una propiedad general del algoritmo BBS.

## 5. RANDU

RANDU es un generador de números pseudoaleatorio desarrollado por IBM, utilizado principalmente durante 1960 y 1970. Perteneció al grupo de generadores congruenciales lineales.

A pesar de su uso masivo, RANDU presenta deficiencias estadísticas, especialmente sesgos y correlaciones entre valores consecutivos. Por estas razones, se considera un ejemplo clásico de un generador pseudoaleatorio mal diseñado, actualmente se encuentra obsoleto.

### Fundamento matemático

RANDU forma parte de los LCG, específicamente es un generador congruencial multiplicativo ya que su incremento es igual a cero.

La ecuación de este generador es un caso particular de la ecuación general de un LCG:

$$X_{n+1} = (a \times X_n + c) \bmod m$$

En RANDU se sustituyen los parámetros de la siguiente manera:

$$a = 65539$$

$$c = 0$$

$$m = 2^{31}$$

Su secuencia se genera por medio de:

$$V_{j+1} = 65539V_j \bmod 2^{31}$$

Donde:

- $V_j$  es el estado actual de la secuencia,
- $V_{j+1}$  es el siguiente estado pseudoaleatorio,
- 65,539 es el multiplicador, también puede representarse como  $2^{16} + 3$ ,
- $2^{31}$  es el módulo, y
- $V_0$  es la semilla inicial.

## 1. Selección de la semilla

RANDU necesita  $V_0$ , a partir de la cual se genera de manera determinista la secuencia. Esta semilla debe ser impar,  $V_0 \bmod 2 = 1$ .

Al general la secuencia, primero se utiliza la semilla para  $V_j$ , luego ese parámetro es cada nuevo valor generado y así poder generar el siguiente valor.

## 2. Normalización

Todos los valores producidos por este generador, son números enteros. Para convertirlos en valores pseudoaleatorios dentro del intervalo  $[0, 1)$  se deben dividir entre el módulo:

$$U_j = \frac{V_j}{2^{31}}$$

Debido a la operación módulo de la recurrencia  $0 \leq V_j < 2^{31}$ , por lo tanto  $0 \leq U_j < 1$ , donde:

- $V_j$  es el valor entero generado por RANDU,
- $2^{31}$  es el módulo del generador, y
- $U_j$  es el valor pseudoaleatorio normalizado.

## Elección del multiplicador

El multiplicador 65539 se eligió debido a la eficiencia computacional que podía obtenerse en las computadoras de la época de los 60s-70s. La multiplicación de  $2^{16}$  podía implementarse eficientemente por medio de operaciones de desplazamiento de bits, por lo cual, lo hacía atractivo para el hardware disponible.

Sin embargo, esta selección de parámetros produjo fuertes dependencias entre los valores generados.

## Dependencia entre valores consecutivos

Una de las deficiencias principales es que los números generados no son suficientemente independientes entre sí. Existe una relación matemática muy sencilla entre tres valores consecutivos de la secuencia.

Como queda:

$$V_{j+2} = (2^{16} + 3)V_j \bmod 2^{31}$$

$$V_{j+1} = (2^{16} + 3)V_j \bmod 2^{31}$$

Se sustituye  $V_{j+1}$  en la primer expresión:

$$V_{j+2} = (2^{16} + 3)((2^{16} + 3)V_j \bmod 2^{31}) \bmod 2^{31} = (2^{16} + 3)^2 V_j \bmod 2^{31}$$

Se desarrolla el cuadrado:

$$V_{j+2} = (2^{32} + 6 \bullet 2^{16} + 9) V_j \bmod 2^{31}$$

Se simplifica la expresión debido a que  $2^{32}$  es divisible exactamente entre el módulo  $2^{31}$  y su residuo es 0:

$$V_{j+2} = (6 \bullet 2^{16} + 9) V_j \bmod 2^{31}$$

Ahora se toma la expresión  $V_{j+1}$  y se multiplica por 6 para tener en ambas expresiones  $6 \bullet 2^{16}$ :

$$6V_{j+1} = (6 \bullet 2^{16} + 18)V_j \bmod 2^{31}$$

Al comparar ambas expresiones, ambas contienen el  $6 \bullet 2^{16}$ , al restar ambas expresiones, este término se elimina:

$$6V_{j+1} - V_{j+2} = ((6 \bullet 2^{16} + 18) - (6 \bullet 2^{16} + 9)) V_j \bmod 2^{31}$$

$$6V_{j+1} - V_{j+2} = (18 - 9)V_j \bmod 2^{31}$$

$$-V_{j+2} = -6V_{j+1} + 9V_j \bmod 2^{31}$$

$$V_{j+2} = 6V_{j+1} - 9V_j \bmod 2^{31}$$

Esto demuestra que tres valores consecutivos de RANDU presentan una fuerte relación lineal, conociendo  $V_j$  y  $V_{j+1}$ , el siguiente valor  $V_{j+2}$  está restringido por esta relación modular. Esta dependencia es la que provoca que al representar ternas consecutivas  $(U_j, U_{j+1}, U_{j+2})$ , los puntos presenten patrones en planos en lugar de distribuirse de manera adecuada en el espacio tridimensional.

## Consecuencia geométrica

La dependencia entre los tres valores consecutivos puede observarse gráficamente formando ternas con los valores normalizados  $(U_j, U_{j+1}, U_{j+2})$ .

En un generador con buenas propiedades estadísticas se esperaría que estas ternas se distribuyeran de manera adecuada a través del espacio tridimensional.

Sin embargo, debido a la relación existente entre los términos consecutivos de RANDU, los puntos generados presentan una estructura evidente y se concentran en un número reducido de planos paralelos en lugar de llenar uniformemente el espacio tridimensional. Esta característica permite visualizar las deficiencias estadísticas de RANDU y constituye una de las razones por las que actualmente se considera un generador inadecuado para simulaciones y aplicaciones que requieren números pseudoaleatorios de buena calidad.

## Pseudocódigo

```
1  Inicio Algoritmo_RANDU
2      Definir semilla, n, i, V como entero
3      Definir U como real
4
5      Repetir
6          Ingresar semilla
7          Si (semilla Mod 2 = 0) Entonces
8              Escribir "La semilla debe ser un numero impar"
9          Fin Si
10         Hasta Que (semilla Mod 2 = 1)
11
12         Ingresar n
13         V = semilla
14
15         Para i = 1 Hasta n Hacer
16             V = (65539 * V) Mod 231
17             U = V / 231
18             Escribir V y U
19         Fin Para
20 Fin Algoritmo_RANDU
```

## Implementación

Se desarrolló el algoritmo RANDU desde cero a partir del pseudocódigo definido anteriormente. Para la implementación se utilizó una semilla inicial impar y se generaron 10,000 valores, y cada valor entero obtenido fue normalizado.

| Parámetro             | Valor    |
|-----------------------|----------|
| Multiplicador $a$     | 65539    |
| Incremento $c$        | 0        |
| Módulo $m$            | $2^{31}$ |
| Semilla               | 8103     |
| Valores generados (n) | 10,000   |

### a. Histograma de los valores generados

Para analizar visualmente la distribución de los valores generados por RANDU, se construyó un histograma utilizando los 10 000 valores normalizados en el intervalo  $[0, 1)$ . Los valores fueron agrupados en 20 intervalos para observar la distribución de sus frecuencias. Bajo una distribución uniforme se esperaría aproximadamente una frecuencia de 500 observaciones por intervalo.

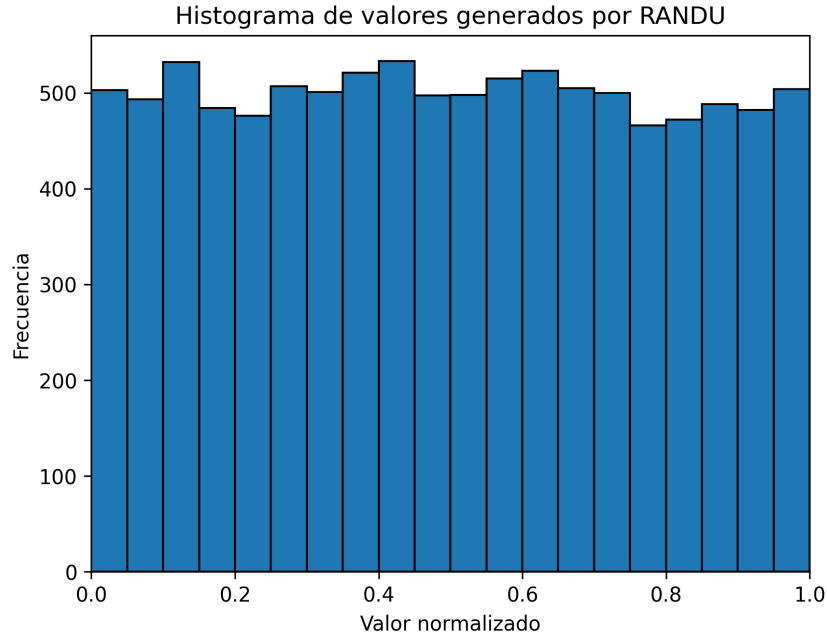


Figura 9: Histograma de valores normalizados generados mediante RANDU.

En el histograma se observa que las frecuencias se mantienen aproximadamente alrededor de las 500 observaciones esperadas por intervalo, sin presentar concentraciones o desviaciones visuales marcadas. Por lo tanto, la representación gráfica sugiere una distribución aproximadamente uniforme de los valores individuales generados.

## b. Pruebas de hipótesis

### b.1. Prueba de uniformidad

Para evaluar formalmente la uniformidad de los valores generados mediante RANDU, se aplicó la prueba de Kolmogorov-Smirnov, utilizando un nivel de significancia de  $\alpha = 0,05$ .

Se establecieron las siguientes hipótesis:

$H_0$ : Los valores generados siguen una distribución uniforme  $U(0,1)$

$H_1$ : Los valores generados no siguen una distribución uniforme  $U(0,1)$

La prueba obtuvo los siguientes resultados  $D = 0,010599$ ,  $p = 0,2097418$ . Debido a que  $p > \alpha$  no se rechaza la hipótesis nula.

Por lo tanto, no existe evidencia estadística suficiente para afirmar que los valores generados mediante RANDU difieren de una distribución uniforme  $U(0,1)$  para la muestra analizada. Este resultado es consistente con la distribución aproximadamente uniforme observada previamente en el histograma.

## b.2. Prueba de independencia serial

Para analizar la independencia entre valores consecutivos se utilizó la autocorrelación de retardo 1 mediante el coeficiente de correlación de Pearson.

Se establecieron las siguientes hipótesis:

$$H_0 : \rho_1 = 0$$

$$H_1 : \rho_1 \neq 0$$

donde  $\rho_1$  representa la correlación entre valores consecutivos de la secuencia.

*Los resultados obtenidos fueron  $r = 0,011363$ ,  $p = 0,2558862$ .* Debido a que  $p > \alpha$ , no se rechaza la hipótesis nula.

Por lo tanto, no existe evidencia estadística suficiente de correlación lineal significativa entre valores consecutivos de la secuencia generada por RANDU. Además, el coeficiente obtenido es cercano a cero, indicando una asociación lineal de retardo 1 muy pequeña en la muestra analizada.

Sin embargo, este resultado únicamente analiza la relación lineal entre pares consecutivos y no descarta la existencia de estructuras de dependencia de mayor dimensión.

## c. Prueba espectral visual

Para analizar posibles patrones estructurales que no pueden identificarse mediante las pruebas anteriores, se construyeron ternas consecutivas de la forma  $(u_i, u_{i+1}, u_{i+2})$  y se presentaron como puntos dentro del cubo tridimensional  $[0, 1]^3$ .

Se obtuvieron 9998 ternas a partir de los 10000 valores generados. Las 9998 ternas fueron distintas, por lo que no se observaron repeticiones dentro de la muestra analizada.

Al observar la representación tridimensional desde un ángulo adecuado, se distinguen franjas correspondientes a planos paralelos sobre los cuales se encuentran las ternas generadas. Por lo tanto, aunque las ternas no se repiten, estas no ocupan libremente todo el espacio tridimensional, sino que presentan una estructura geométrica definida.

Este comportamiento puede analizarse mediante la relación característica de RANDU:

$$V_{i+2} \equiv 6V_{i+1} - 9V_i \pmod{2^{31}}$$

Al trabajar con los valores normalizados, las ternas satisfacen una relación de la forma:

$$u_{i+2} - 6u_{i+1} + 9u_i = k$$

donde  $k$  toma valores enteros. Para verificar la estructura observada visualmente, se calcularon los valores de  $k$  correspondientes a las ternas generadas, obteniéndose:

$$k \in \{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Esto corresponde a 15 planos paralelos.

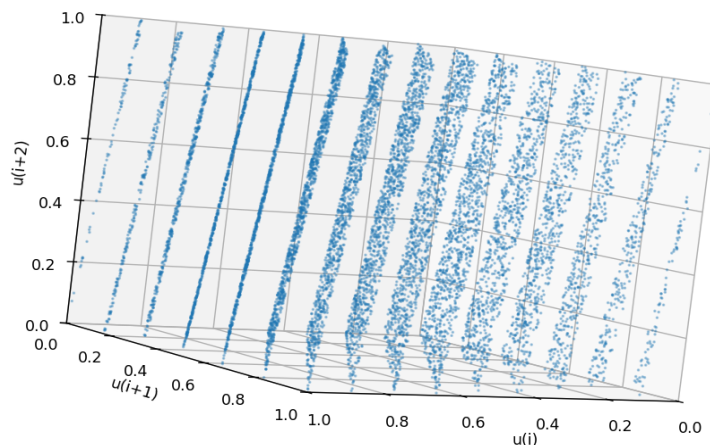


Figura 10: Prueba espectral visual de las ternas consecutivas generadas mediante RANDU.

Por lo tanto, la prueba espectral evidencia una deficiencia importante de RANDU, a pesar de que el histograma presenta una distribución visualmente uniforme, la prueba de Kolmogorov-Smirnov no rechaza la uniformidad y la autocorrelación de retardo 1 no detecta una correlación lineal significativa, las ternas consecutivas presentan una estructura geométrica claramente definida.

Este resultado muestra que superar pruebas individuales de uniformidad e independencia serial no garantiza por sí solo una buena calidad del generador. En RANDU, la estructura observada en tres dimensiones evidencia dependencia entre valores consecutivos que no fue detectada mediante las pruebas anteriores.

Finalmente, aunque el generador posee un período relativamente amplio, la presencia de esta estructura demuestra que un período largo no garantiza una buena calidad estadística, ya que los puntos generados pueden presentar patrones deterministas en dimensiones superiores.

## Comparación con las librerías estándar random y secrets de Python

Para complementar el análisis de los generadores pseudoaleatorios, se seleccionó Mersenne Twister (MT19937) para realizar una comparación con las librerías estándar random y secrets de Python.

La comparación se realizó utilizando 10,000 valores normalizados en el intervalo  $[0, 1)$  para cada alternativa. Para mantener la reproducibilidad del experimento, la implementación

propia de MT19937 y la librería random utilizaron la semilla 8103. En el caso de secrets, no se estableció una semilla manual, debido a que esta librería utiliza una fuente de aleatoriedad proporcionada por el sistema operativo.

Se consideraron tres fuentes de números aleatorios o pseudoaleatorios: la implementación propia de MT19937 desarrollada anteriormente a partir del pseudocódigo, la librería estándar random de Python y la librería estándar secrets. Para realizar la comparación se evaluaron el tiempo requerido para generar los valores, la uniformidad de la secuencia y la independencia serial entre valores consecutivos.

Los resultados obtenidos se presentan en la siguiente tabla:

| Generador | Tiempo de generación | Estadístico K-S | p-valor K-S | Correlación retardo 1 | p-valor correlación |
|-----------|----------------------|-----------------|-------------|-----------------------|---------------------|
| MT19937   | 0.01847040           | 0.007441        | 0.634227    | 0.005389              | 0.590031            |
| Random    | 0.00142210           | 0.005352        | 0.935442    | 0.005112              | 0.609298            |
| Secrets   | 0.00550820           | 0.010522        | 0.216663    | -0.004249             | 0.670955            |

Tabla 1. Comparación entre la implementación propia de MT19937 y las librerías estándar random y secrets de Python.

Adicionalmente, se construyeron histogramas de los valores obtenidos con cada alternativa, con el propósito de comparar visualmente su distribución en el intervalo  $[0, 1)$ .

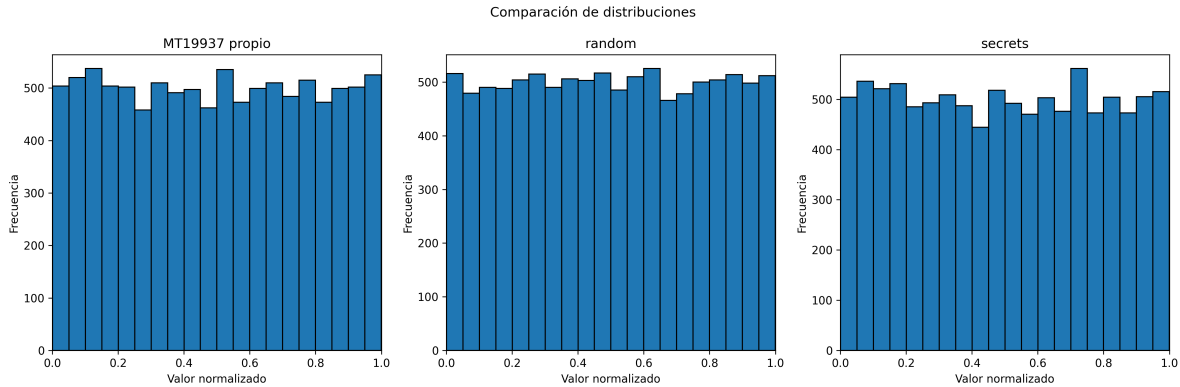


Figura 11: Comparación de los histogramas de valores generados mediante MT19937 propio, random y secrets.

A partir de los histogramas y de los resultados obtenidos mediante la prueba de Kolmogorov-Smirnov, se analizó si las secuencias generadas presentan un comportamiento compatible con una distribución uniforme  $U(0, 1)$ . Asimismo, mediante la autocorrelación de retardo 1 se evaluó la existencia de correlación lineal significativa entre valores consecutivos.

Los resultados de la prueba de Kolmogorov-Smirnov muestran que los tres métodos obtuvieron p-valores superiores al nivel de significancia  $\alpha = 0,05$ . Por lo tanto, en ninguno de los casos se rechaza la hipótesis nula de uniformidad. Este resultado es consistente con los histogramas obtenidos, en los cuales las frecuencias se distribuyen de manera relativamente uniforme a lo largo del intervalo  $[0, 1)$ .

En la prueba de independencia serial, los tres métodos presentaron coeficientes de correlación cercanos a cero y p-valores superiores a 0,05. Por lo tanto, no se rechaza la hipótesis nula  $H_0 : \rho_1 = 0$ , indicando que no existe evidencia estadística de correlación lineal significativa entre valores consecutivos en las secuencias analizadas.

Respecto al tiempo de generación, random presentó el menor tiempo, con 0.00142210 segundos, seguido de secrets, con 0.00550820 segundos, y finalmente la implementación propia de MT19937, con 0.01847040 segundos. Estos valores corresponden a la ejecución realizada y pueden variar según el entorno y las condiciones de ejecución.

Aunque las tres alternativas permiten generar valores con apariencia aleatoria, existen diferencias importantes respecto a su propósito. La implementación propia de MT19937 permite observar directamente el funcionamiento interno del algoritmo y establecer una semilla para reproducir una secuencia determinada. De manera similar, random permite establecer una semilla, característica que facilita la reproducción de experimentos y simulaciones.

Por otro lado, secrets está orientada a situaciones en las que la impredecibilidad de los valores generados es una característica importante. A diferencia de los dos casos anteriores, no se utiliza estableciendo una semilla manual para reproducir una secuencia, sino que obtiene su aleatoriedad de fuentes proporcionadas por el sistema operativo.

En consecuencia, la selección de un generador depende de las necesidades de la aplicación. Tanto la implementación propia de MT19937 como la librería random permiten trabajar con semillas y reproducir secuencias, lo cual resulta útil en simulaciones y experimentos computacionales. Por otro lado, secrets está orientada a aplicaciones en las que la impredecibilidad y la seguridad son prioritarias.

Esta comparación también permite observar que la calidad de un generador no debe evaluarse únicamente a partir de la apariencia de sus valores o de una sola prueba estadística. Además de la uniformidad y la independencia de las secuencias, deben considerarse características como la velocidad, la reproducibilidad, la seguridad y el propósito para el cual fue diseñado.

## Tabla de comparación

| Generador | Velocidad                                                                                          | Período                                                | Seguridad                            | Casos de uso                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LCG       | Sumamente veloz por utilizar operaciones aritméticas básicas a nivel de procesador por cada ciclo. | Números generados antes de repetir la secuencia exacta | Inseguro por su algoritmo predecible | <ul style="list-style-type: none"> <li>■ Videojuegos: por el concepto de semilla.</li> <li>■ Simulaciones estadísticas rápidas: método de Montecarlo por las tendencias masivas y la gran cantidad de muestras en pocos segundos.</li> <li>■ Sistemas embebidos y microcontroladores: utiliza muy poco código y consume pocos bytes de memoria, esencial para hardware limitado.</li> <li>■ Testing: permite rellenar bases de datos demasiado rápido y permite utilizar la misma semilla para replicar errores y poder corregirlos.</li> </ul> |

| Generador            | Velocidad                                                                         | Período                                                                                                                                                                                                                                                                  | Seguridad                                                                                                                                                                                                                                                                                                                                                                                                         | Casos de uso                                                                                                                                                                                                                                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Middle-square method | Sumamente veloz por su uso de operaciones elementales de bajo costo por iteración | <ul style="list-style-type: none"> <li>■ Impredecible y corto</li> <li>■ Depende de la semilla</li> <li>■ Es fragmentado</li> <li>■ Matemáticamente inestable debido a que colapsa (a cero o números fijos) y destruye la aleatoriedad.</li> <li>■ Es pequeño</li> </ul> | <ul style="list-style-type: none"> <li>■ Nada seguro ya que puede calcular los números siguientes al interceptar o conocer el valor actual generado.</li> <li>■ Genera patrones predecibles, por lo que se puede predecir qué números tienen mayor probabilidad de aparecer.</li> <li>■ Se puede forzar el colapso al estado cero y romper la seguridad de forma masiva en un sistema (ejemplo tokens)</li> </ul> | <ul style="list-style-type: none"> <li>■ Hardware antiguo o chips de juguetes de muy bajo costo, si se necesitan pocos números pseudoaleatorios y el colapso a cero no afecta la experiencia.</li> <li>■ Enseñanza de generadores de números aleatorios.</li> <li>■ Hashing básico para calcular posiciones en tablas de datos.</li> </ul> |

| Generador        | Velocidad                    | Período                                                                                                                                                                                                                                                                                                           | Seguridad                                                                                                                                                                                                                                                                                                                                                                   | Casos de uso                                                                                                                                                                                                                                                                      |
|------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mersenne Twister | Alta velocidad de ejecución. | <ul style="list-style-type: none"> <li>■ Es largo ya que es de <math>2^{19937} - 1</math> (cifra de 6,002 dígitos)</li> <li>■ No existen patrones visibles en secuencias largas debido a la equidistribución.</li> </ul>                                                                                          | <ul style="list-style-type: none"> <li>■ Nada seguro para criptografía, es puramente estadístico el generador.</li> <li>■ Es fácil de revertir con ecuaciones lineales del algoritmo.</li> <li>■ Se puede predecir el 100 % de los números siguientes que generará el sistema cuando se tiene resuelto el estado, o calcular los números generados en el pasado.</li> </ul> | <ul style="list-style-type: none"> <li>■ Simulaciones donde son necesarios millones de datos sin sesgos estadísticos.</li> <li>■ Generación masiva de datos sintéticos para pruebas de estrés.</li> <li>■ Desarrollo de software interactivo y motores de renderizado.</li> </ul> |
| BBS              | Muy lento                    | <ul style="list-style-type: none"> <li>■ Depende de los números primos iniciales, aunque se considera un periodo gigante.</li> <li>■ No es fijo ya que depende de la semilla.</li> <li>■ Si <math>p</math> y <math>q</math> se seleccionan mal, entonces su órbita es corta, destruyendo su seguridad.</li> </ul> | <ul style="list-style-type: none"> <li>■ Criptográficamente seguro.</li> <li>■ Si se descubren los bits actuales, no se puede adivinar los bits futuros ni los pasados.</li> </ul>                                                                                                                                                                                          | <ul style="list-style-type: none"> <li>■ Generar llaves</li> <li>■ Firmas digitales de alta seguridad</li> <li>■ Estándar de referencias en investigación académica.</li> </ul>                                                                                                   |

| Generador | Velocidad                                                                                               | Período                                                                                                                                                                                                                                                                                                         | Seguridad                                                                                                                                                                               | Casos de uso                                                                                                                                                                                                                                                                  |
|-----------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RANDU     | Extremadamente rápido, logrado mediante desplazamiento de bits y sumas con un multiplicador específico. | <ul style="list-style-type: none"> <li>■ Tiene período de <math>2^{29}</math>, siendo una longitud extremadamente corta para la computación actual.</li> <li>■ Al ser un generador multiplicativo con módulo <math>2^{31}</math>, su ciclo se restringe a <math>\frac{1}{4}</math> parte del módulo.</li> </ul> | <p>Seguridad nula por:</p> <ul style="list-style-type: none"> <li>■ Ser completamente predecible</li> <li>■ Correlación lineal extrema</li> <li>■ Espacio de claves diminuto</li> </ul> | <ul style="list-style-type: none"> <li>■ No tiene aplicación práctica actualmente</li> <li>■ Replicación de simulaciones científicas antiguas.</li> <li>■ Pruebas de aleatoriedad para verificar que las herramientas detecten correctamente patrones defectuosos.</li> </ul> |

## Referencias

1. [https://en.wikipedia.org/wiki/Linear\\_congruential\\_generator](https://en.wikipedia.org/wiki/Linear_congruential_generator)
2. <https://docentefelipegutierrezcerda.blogspot.com/2026/07/generador-lineal-congruencial.html>
3. <https://medium.com/@vayadanderightna/understanding-the-linear-congruential-generator-lcg-40e2a24b2a67>
4. <https://qualityandinnovation.com/2015/03/03/a-linear-congruential-generator-lcg-in-r/>
5. <https://rpubs.com/aaronsc32/linear-congruential-generator-r>
6. [https://en.wikipedia.org/wiki/Middle-square\\_method](https://en.wikipedia.org/wiki/Middle-square_method)
7. <https://medium.com/@vayadanderightna/how-the-middle-square-method-works-for-random-number-generation-395079879b67>
8. [https://www.maxime-bros.fr/files/teaching/reseau/linear\\_prng.pdf](https://www.maxime-bros.fr/files/teaching/reseau/linear_prng.pdf)
9. <https://www.math.cmu.edu/~af1p/Texfiles/LINEARCONGRU.pdf>
10. <https://www.gamedeveloper.com/programming/how-classic-games-make-smart-use-of-random-number-generation>
11. <https://www.sciencedirect.com/topics/computer-science/merzenne-twister>
12. [https://es.wikipedia.org/wiki/N%C3%BAmero\\_primo\\_de\\_Mersenne](https://es.wikipedia.org/wiki/N%C3%BAmero_primo_de_Mersenne)
13. <https://dl.acm.org/doi/10.1145/272991.272995>
14. <https://blogs.mathworks.com/cleve/2015/04/17/random-number-generator-merzenne-twister/>
15. [https://en.wikipedia.org/wiki/Mersenne\\_Twister](https://en.wikipedia.org/wiki/Mersenne_Twister)
16. [https://oneapi-spec.uxlfoundation.org/specifications/oneapi/v1.4-rev-1/elements/onemath/source/domains/rng/host\\_api/onemath-rng-mt19937](https://oneapi-spec.uxlfoundation.org/specifications/oneapi/v1.4-rev-1/elements/onemath/source/domains/rng/host_api/onemath-rng-mt19937)
17. <https://www.sciencedirect.com/topics/computer-science/merzenne-twister>
18. <https://www.math.sci.hiroshima-u.ac.jp/m-mat/MT/ARTICLES/sfmt.pdf>
19. <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-vector-statistics-notes/2021-1/mt19937.html>
20. [https://en.wikipedia.org/wiki/Blum\\_Blum\\_Shub](https://en.wikipedia.org/wiki/Blum_Blum_Shub)

21. <https://crypto.stackexchange.com/questions/109081/period-of-blum-blum-s-hub>
22. <https://www.semanticscholar.org/topic/Blum-Blum-Shub/804859>
23. <https://homes.luddy.indiana.edu/kapadia/project2/node11.html>
24. <https://en.wikipedia.org/wiki/RANDU>
25. <https://randu.me/pseudorandom/>
26. <https://cmd.inp.nsk.su/dp6s/numerics/randu/randu.pdf>
27. <https://webs.um.es/mpulido/miwiki/lib/exe/fetch.php?id=amio&cache=cach&media=wiki:simt1.pdf>